Machine Learning — Lecture 6

Decision Trees and K-Nearest Neighbors

FCIS — For Course Use Only

Dr. Fayçal Hamdi

Faculty of Computer and Information Systems

Lecture 6 at a Glance

FCIS — Internal Use Only

Decision Trees and K-Nearest Neighbors

Course Learning Outcomes

CLO3: Construct and analyze machine learning models by applying algorithmic principles and internal computations.

CLO4: Apply supervised learning techniques to solve practical problems and interpret their outcomes.

Lecture purpose

- Introduce models that learn differently from regression-based models.
- Explain how decision trees choose splits.
- Compute entropy, Gini impurity, and information gain.
- Analyze tree complexity, pruning, and interpretability.
- Use KNN as an instance-based, distance-based classifier.
- Connect preprocessing and evaluation to tree and KNN behavior.

FCIS — For Course Use Only

💡 Main idea: not all models learn weights by gradient descent. Decision trees learn **rules**, while KNN predicts from **nearby examples**.

Learning Objectives

FCIS — Course Use Only

By the end of Lecture 6, you will be able to:

1. Explain how a decision tree represents a prediction rule.
2. Compute entropy and Gini impurity for a node.
3. Compute **information gain** for a candidate split.
4. Build a small decision tree manually from training examples.
5. Explain how tree depth affects underfitting and overfitting.
6. Interpret decision-tree predictions and feature importance cautiously.
7. Explain K-Nearest Neighbors as instance-based learning.
8. Compute Euclidean distance and perform a KNN prediction.
9. Explain why scaling strongly affects KNN.
10. Compare trees and KNN in terms of assumptions, interpretability, and evaluation.

CCIS — For Course Use Only

Lecture Roadmap

FCIS — Course Use Only

From decision rules to neighborhood-based learning

Learning phase	Parts	Core question and student outcome
1. Understand tree predictions	**I–II**	How does a tree follow feature tests? Trace a root-to-leaf prediction.
2. Measure and build splits	**III–V**	Which split reduces uncertainty most? Compute impurity and construct a small tree.
3. Control tree complexity	**VI**	Why do deep trees overfit? Interpret validation curves and pruning controls.
4. Learn from nearby examples	**VII–VIII**	How do k, distance, and scaling affect KNN? Compare KNN and trees.

FCIS — [illegible]

💡 **Practice & consolidation:** reason through split selection, neighbor votes, scaling, and validation without using the test set for tuning.

Part I

FCIS — For Students Use Only

Why Study These Models?

Regression and logistic regression learn numerical weights. **Decision trees** and KNN show two different ways to learn from data.

FCIS — For Course Use Only

Three Learning Philosophies

FCIS — Private Use Only

A useful comparison

Model family	What is learned?	Prediction style
Linear / Logistic Regression	**coefficients**	compute a weighted score
Decision Trees	**splitting rules**	follow a path of decisions
K-Nearest Neighbors	**stored training examples**	compare to nearby examples

Why this matters

Different algorithms impose **different assumptions**.

- **Linear models** assume a simple functional form.
- **Trees** learn if-then rules and partitions.
- **KNN** assumes nearby points should have similar labels.

FCIS - For Course Use Only

💡 A good ML practitioner understands not only **how to run a model**, but also **what kind of pattern it can represent**.

Where They Fit in the Course

FCIS_ For Students Use Only

From formulas to rules and neighbors

What we already know

Previous lectures introduced **features and preprocessing**, **linear and logistic regression**, **loss functions and gradient descent**, and **model evaluation**.

A new question

Can we build **predictive** models **without using gradient descent?**

Decision Trees — Learn Rules

Recursive splitting: Divide the data into groups using feature-based decisions.

Prediction: Follow a sequence of *if–then* rules.

KNN — Use Neighbors

Similarity: Find the closest stored training examples.

Prediction: Use their labels or target values to make a prediction.

FCIS — For Course Use Only

> **Key idea.** Decision Trees learn **decision rules**, while KNN predicts using **similarity to training examples**. Neither requires gradient descent.

Part II

FCIS — For Private Use Only

Decision Trees

A decision tree predicts by following **a sequence of feature-based tests** from root to leaf.

FCIS — For Course Use Only

What is a Decision Tree?

FCIS — Personal Use Only

A hierarchy of questions

A **decision tree** contains:

Component	Role
Root	First decision applied to all observations
Internal node	Additional feature test
Branch	Outcome of a test
Leaf	Final prediction
Depth	Maximum number of edges from root to leaf

FCIS — For Course Use Only

Example rule path

```
if attendance < 70:
    predict at-risk
else if missed_labs > 3:
    predict at-risk
else:
    predict not at-risk
```

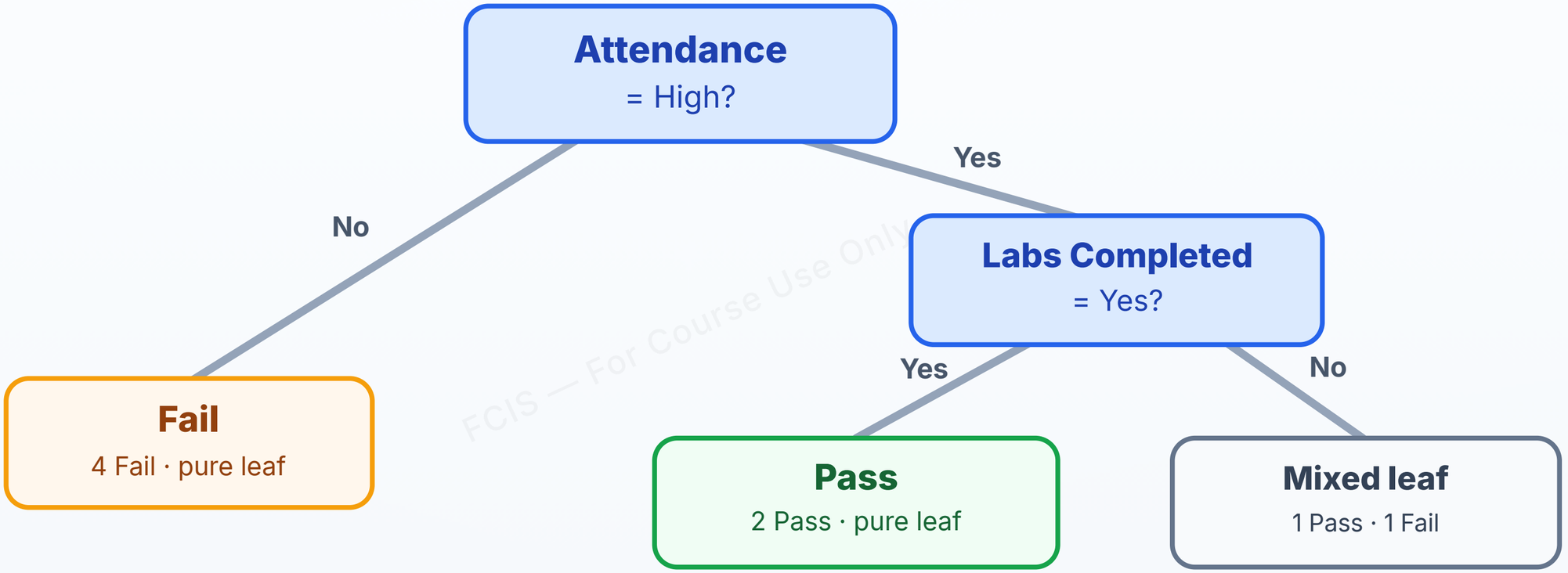
💡 **Key idea.** A decision tree is interpretable because each prediction can be traced through a **sequence of decisions from root to leaf**.

Visual: Follow a Decision-Tree Path

FCIS — For Classroom Use Only

Rules become a prediction when we follow the branches

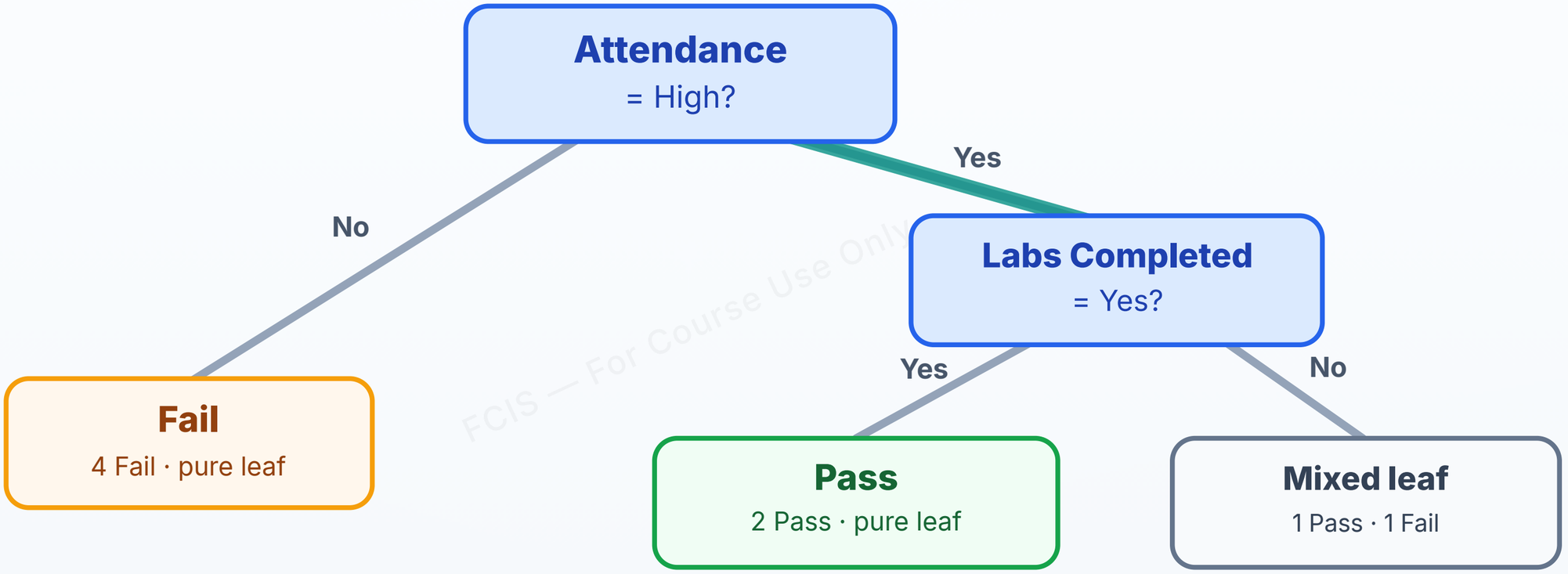
Example student: Attendance = **High**; Labs Completed = **Yes**. Click to follow the two decisions.



FCIS — For Course Use Only

Rules become a prediction when we follow the branches

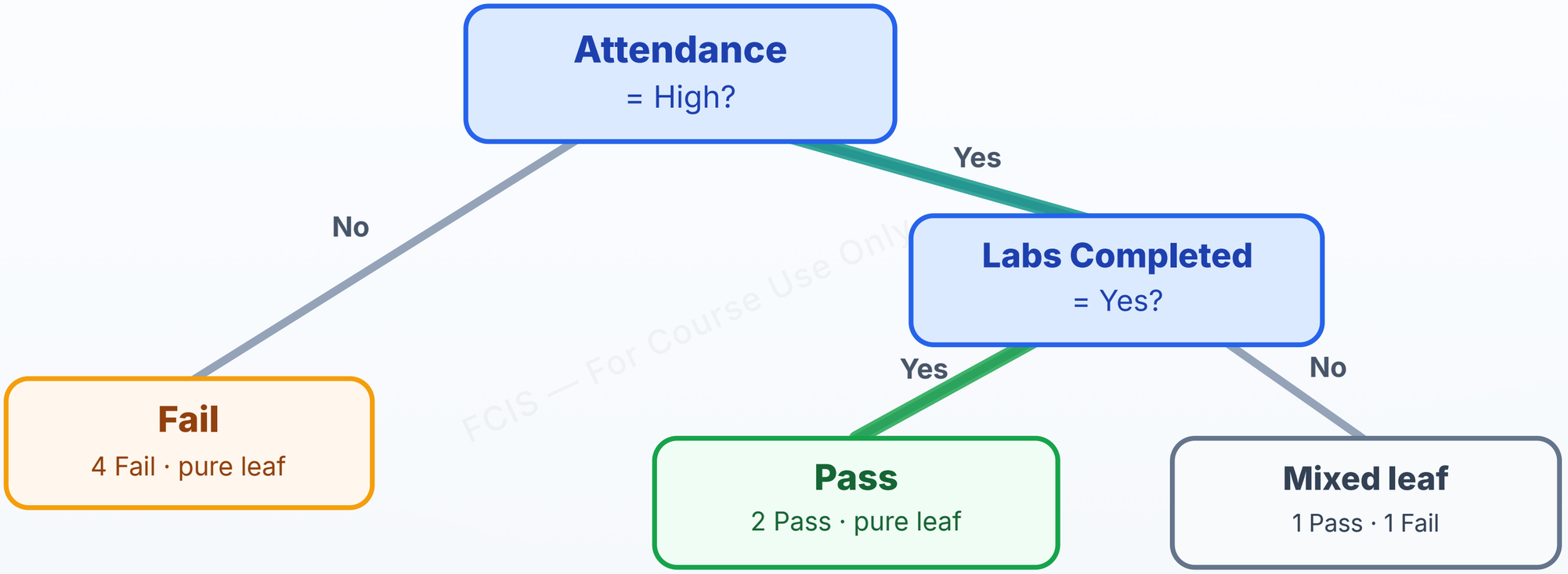
Example student: Attendance = **High**; Labs Completed = **Yes**. Click to follow the two decisions.



FCIS — For Course Use Only

Rules become a prediction when we follow the branches

Example student: Attendance = **High**; Labs Completed = **Yes**. Click to follow the two decisions.



FCIS — For Course Use Only

Prediction: Pass. The student reaches a pure leaf containing 2 Pass examples.

Classification Tree Prediction

FCIS — For Course Use Only

Leaf majority class

In a classification tree:

1. start at the root;
2. apply the feature test;
3. move to the corresponding child;
4. repeat until a leaf is reached;
5. predict the **majority class** in the leaf.

A leaf may also estimate class probabilities from training class frequencies (these estimates are not necessarily well calibrated):

FCIS — For Course Use Only

$$P(y = k \mid \text{leaf}) = \frac{\text{number of class } k \text{ examples in leaf}}{\text{number of examples in leaf}}$$

> If a leaf contains 8 pass and 2 fail students, it may predict pass with probability 0.8.

Regression Trees

FCIS — Private Use Only

Same structure, numerical prediction

For **regression** trees, the path structure is the same.

But a leaf predicts a number, usually the mean target value of training examples in that leaf:

$$\hat{y}_{\text{leaf}} = \frac{1}{m} \sum_{i=1}^{m} y_i$$

Example

If a leaf contains rents:

$$1800,\ 2000,\ 2200$$

FCIS - For Course Use Only

then the leaf prediction is:

$$\hat{y} = 2000$$

> This lecture focuses mainly on classification trees, but the same splitting idea extends to regression.

Why Trees Are Useful

FCIS — Lecture Use Only

Strengths

Decision trees are useful because they:

- **Capture nonlinear relationships** without specifying a mathematical equation.
- **Model feature interactions** through successive splits.
- Are easy to **visualize and explain** when reasonably small.
- Produce **human-readable decision rules**.
- Generally **do not require feature scaling**, unlike distance-based models.
- Can handle **numerical and categorical features** (with appropriate encoding or algorithm support).

FCIS — For Course Use Only

Example: Feature interaction

A tree can learn a rule such as:

> **IF** income is high **AND** debt is high, **THEN** predict high financial risk.

> 💡 **Key idea.** Decision trees learn **interpretable, nonlinear rules** and can capture feature interactions without manually creating interaction features.

Why Trees Can Fail

FCIS — Course Use Only

Limitations

Despite their flexibility, decision trees have important limitations:

- **Overfitting:** Deep trees may memorize training details.
- **Instability:** Small data changes can produce different tree structures.
- **Complexity:** Large trees become difficult to interpret.
- **Split-selection bias:** Some methods favor features with many candidate splits.
- **Probability estimates:** Small leaves can produce unreliable probabilities.
- **Smooth relationships:** Approximating continuous trends may require many splits.

How can we reduce these problems?

Control tree complexity through **maximum depth**, **minimum leaf size**, or **pruning**, and select settings using validation data.

> **Key lesson.** A decision tree is most interpretable when it remains **reasonably small**. Greater complexity does not necessarily improve generalization.

FCIS — For Course Use Only

Part III

FCIS — For Private Use Only

Node Impurity

A split is useful when it makes the child nodes more pure than the parent node.

FCIS — For Course Use Only

What is Impurity?

FCIS — For Course Use Only

Mixed labels inside a node

A node is **pure** if all examples in it have the same class.

A node is **impure** if it contains a mixture of classes.

Example

Node content	Interpretation
10 pass, 0 fail	pure
5 pass, 5 fail	highly impure
9 pass, 1 fail	mostly pure

FCIS — For Course Use Only

A decision tree chooses splits that reduce **impurity**.

> Tree learning is a greedy process: at each node, choose the split that looks best locally.

Entropy

FCIS — Personal Use Only

Measuring class uncertainty

For a node with class probabilities $p_1, \ldots, p_K$, **entropy** is (with $0 \log_2 0 = 0$):

$$H(S) = -\sum_{k=1}^{K} p_k \log_2(p_k)$$

For binary classification:

$$H(S) = -p_+ \log_2(p_+) - p_- \log_2(p_-)$$

FCIS — For Course Use Only

Interpretation

Node	Entropy
pure	0
evenly mixed binary node	1
partially mixed	between 0 and 1

Entropy Worked Example

FCIS — Course Use Only

A node with 6 pass and 4 fail

Suppose a decision-tree node contains **10 examples**:

- **6 Pass** and **4 Fail**
- Class proportions: $p_{\text{pass}} = 0.6$ and $p_{\text{fail}} = 0.4$

Calculate entropy

$$H(S) = -\left[0.6 \log_2(0.6) + 0.4 \log_2(0.4)\right]$$

$$\approx \boxed{0.971 \text{ bits}}$$

FCIS — For Course Use Only

Interpretation

For two classes:

- **Entropy = 0:** Pure node (all examples belong to one class).
- **Entropy = 1:** Maximum uncertainty (50% Pass, 50% Fail).

Conclusion. With 60% Pass and 40% Fail, the node is **fairly mixed**. Its entropy (**0.971**) is close to the maximum of **1 bit**.

Gini Impurity

FCIS — Private Use Only

Another measure of node impurity

Gini impurity measures how mixed the class labels are in a decision-tree node.

For K classes, where p_k is the proportion of class k:

$$Gini(S) = 1 - \sum_{k=1}^{K} p_k^2$$

For **binary classification**:

$$Gini(S) = 1 - p_+^2 - p_-^2$$

FCIS — For Course Use Only

Interpretation

Binary node	Gini impurity
Pure (100% / 0%)	0
Partially mixed	Between 0 and 0.5
Evenly mixed (50% / 50%)	0.5

Key idea. Lower Gini means a **purer node**. Like entropy, Gini measures class mixture, but the two measures use different numerical scales.

Gini Worked Example

FCIS — Internal Use Only

Same node: 6 pass and 4 fail

Recall our node containing **10 examples**:

- **6 Pass** and **4 Fail**
- Class proportions: $p_{\text{pass}} = 0.6$ and $p_{\text{fail}} = 0.4$

Calculate Gini impurity

$$\begin{aligned} Gini(S) &= 1 - \left(0.6^2 + 0.4^2\right) \\ &= 1 - 0.36 - 0.16 \\ &= \boxed{0.48} \end{aligned}$$

FCIS — For Course Use Only

Interpretation

The result is close to the **maximum binary Gini impurity of 0.5**.

> **Conclusion.** The node is **quite mixed**: 60% Pass and 40% Fail. Its Gini impurity (**0.48**) is close to that of a perfectly balanced binary node (**0.5**).

Entropy vs. Gini

FCIS _ Private Use Only

Similar purpose, different formulas

Both measures quantify **class impurity** in a decision-tree node.

Aspect	Entropy	Gini
Formula	$-\sum_k p_k \log_2 p_k$	$1 - \sum_k p_k^2$
Pure node	0	0
Binary maximum	1 bit	0.5
Interpretation	Class uncertainty	Class mixture
Calculation	Uses logarithms	Uses squared probabilities

FCIS — For Course Use Only

For our 60% Pass / 40% Fail node

$$\boxed{H(S) \approx 0.971 \text{ bits}} \qquad \boxed{Gini(S) = 0.48}$$

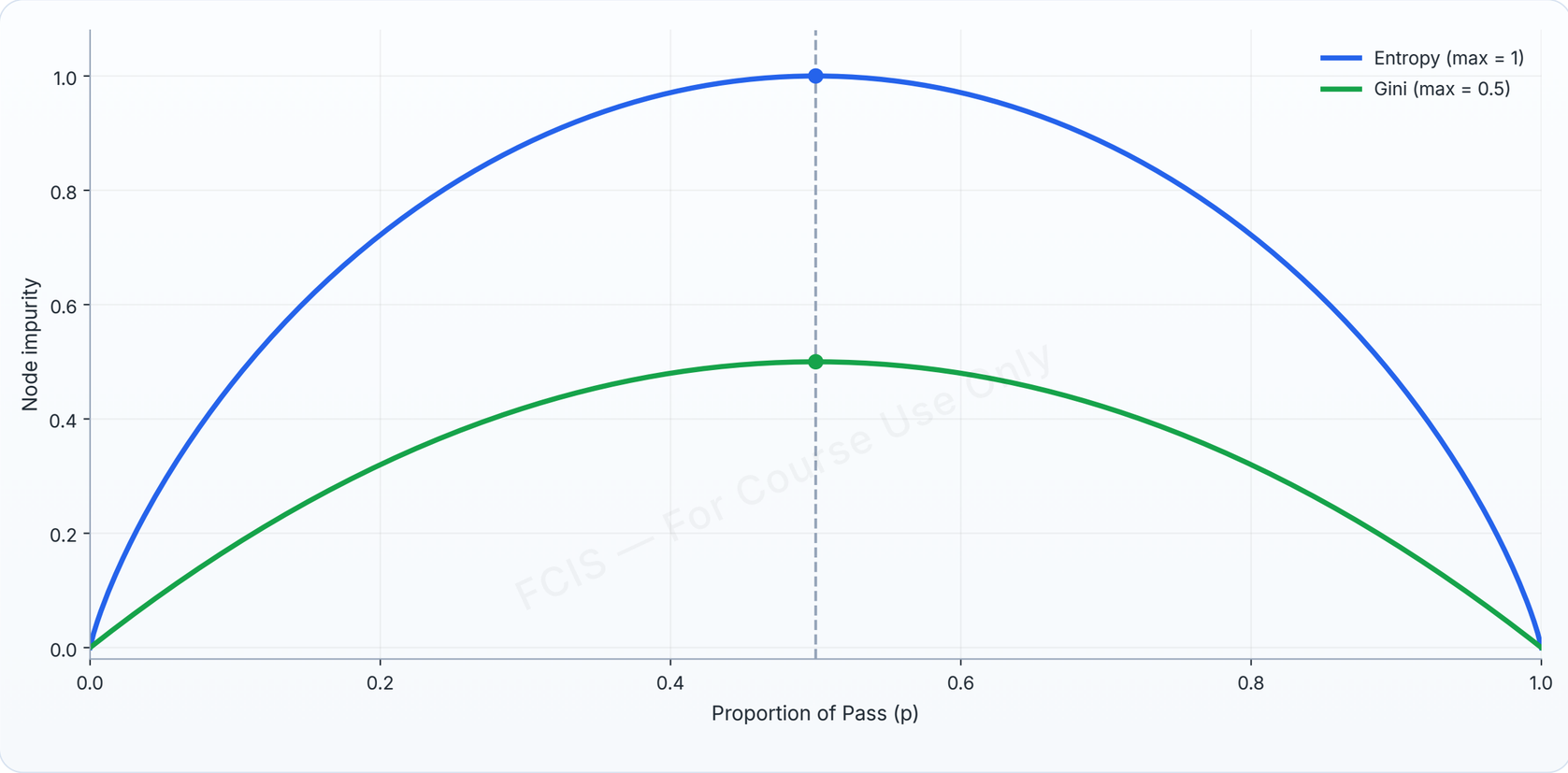
💡 **Key idea.** Both measures are **0 for pure nodes** and highest when the classes are equally balanced. Decision trees favor splits that reduce **weighted child impurity**, using one criterion consistently.

Plot: Entropy and Gini Across Class Balance

For Course Only

How impurity changes with class proportions

For a binary node, let p be the proportion of **Pass** and $1 - p$ the proportion of **Fail**.



💡 **Key idea.** Both curves peak at a **50–50 class split** and fall to zero for pure nodes. Their numerical scales differ, so compare impurity reductions **within the same criterion**.

Part IV

FCIS — Personal Use Only

Information Gain

Information gain measures how much a split reduces uncertainty.

FCIS — For Course Use Only

Weighted Child Impurity

FCIS — Academic Use Only

A split creates child nodes

Suppose a parent node S is split into children:

$$S_1, S_2, \ldots, S_m$$

The weighted **impurity** after the split is:

$$\sum_{j=1}^{m} \frac{|S_j|}{|S|} Impurity(S_j)$$

The weights matter because larger child nodes represent more examples.

> A tiny pure child does not make a split good if the large child remains highly impure.

For Course Use Only

Information Gain Formula

FCIS — For Course Use Only

Entropy reduction

Information gain using entropy is:

$$IG(S, A) = H(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} H(S_v)$$

Where:

- S is the parent dataset at the node;
- A is the feature or split being considered;
- S_v is the subset produced by value or branch v.

For — For Course Use Only

Selection rule

Choose the split with **largest information gain**.

Split Candidate Example

FCIS — Course Use Only

Parent and two children

A candidate split divides a parent node into two children:

Node	Pass	Fail	Total	Entropy
Parent (S)	6	4	10	0.971
Left (L)	4	1	5	0.722
Right (R)	2	3	5	0.971

Child entropy calculations

For Course Use Only

$$H(L) = -\left[\frac{4}{5}\log_2\frac{4}{5} + \frac{1}{5}\log_2\frac{1}{5}\right] \approx 0.722$$

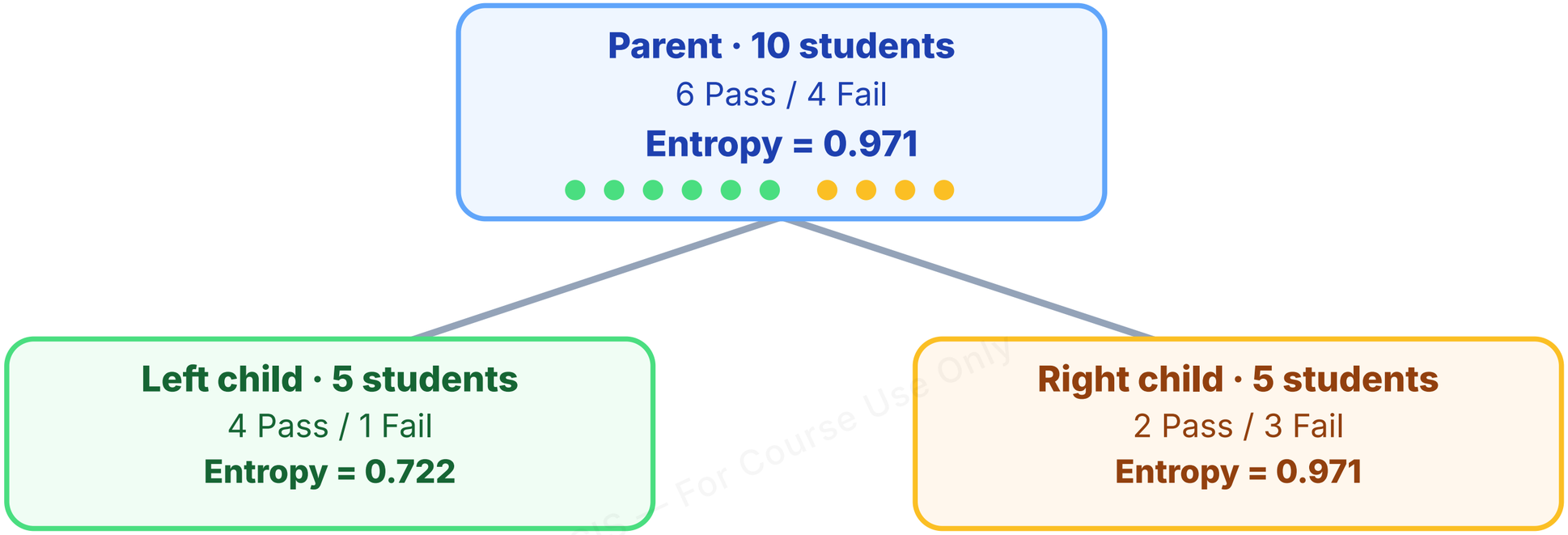
$$H(R) = -\left[\frac{2}{5}\log_2\frac{2}{5} + \frac{3}{5}\log_2\frac{3}{5}\right] \approx 0.971$$

Parent entropy: $H(S) \approx 0.971$

Visual: What Makes a Split Useful?

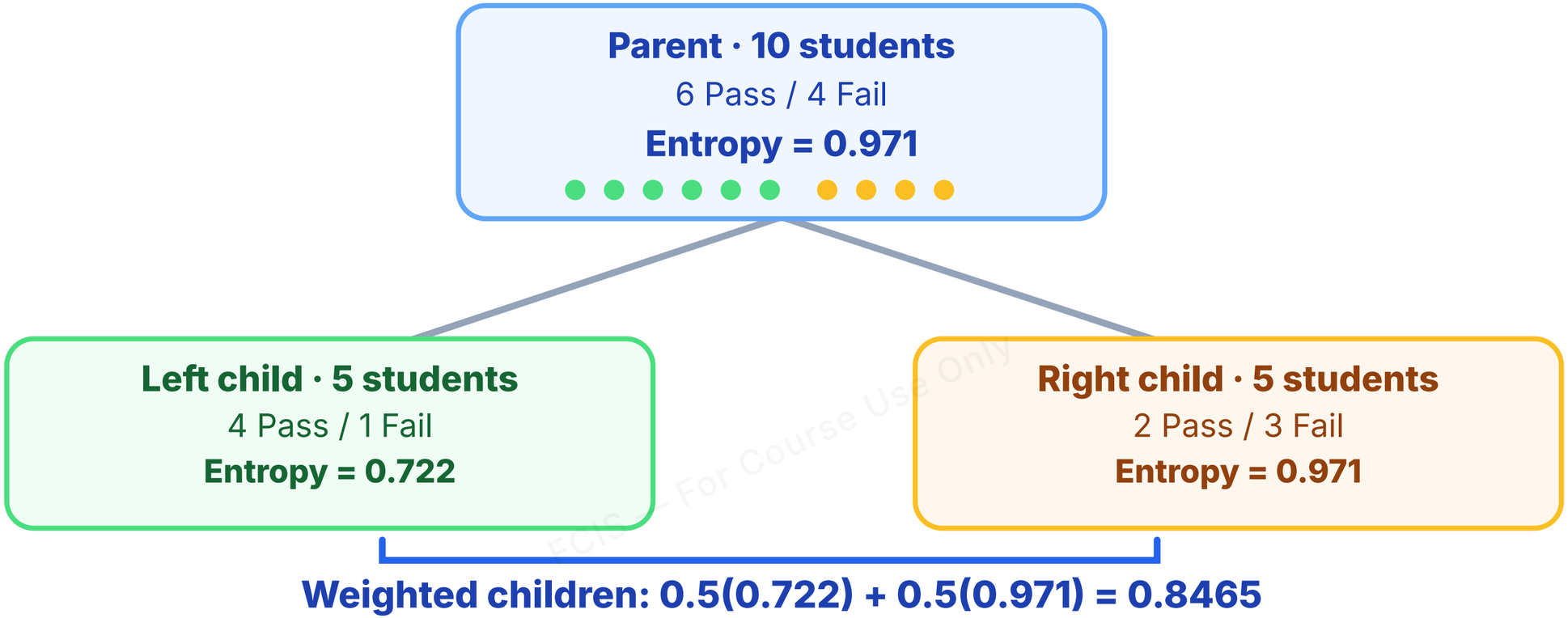
FCIS — Course Use Only

Compare the parent to the weighted impurities of its children



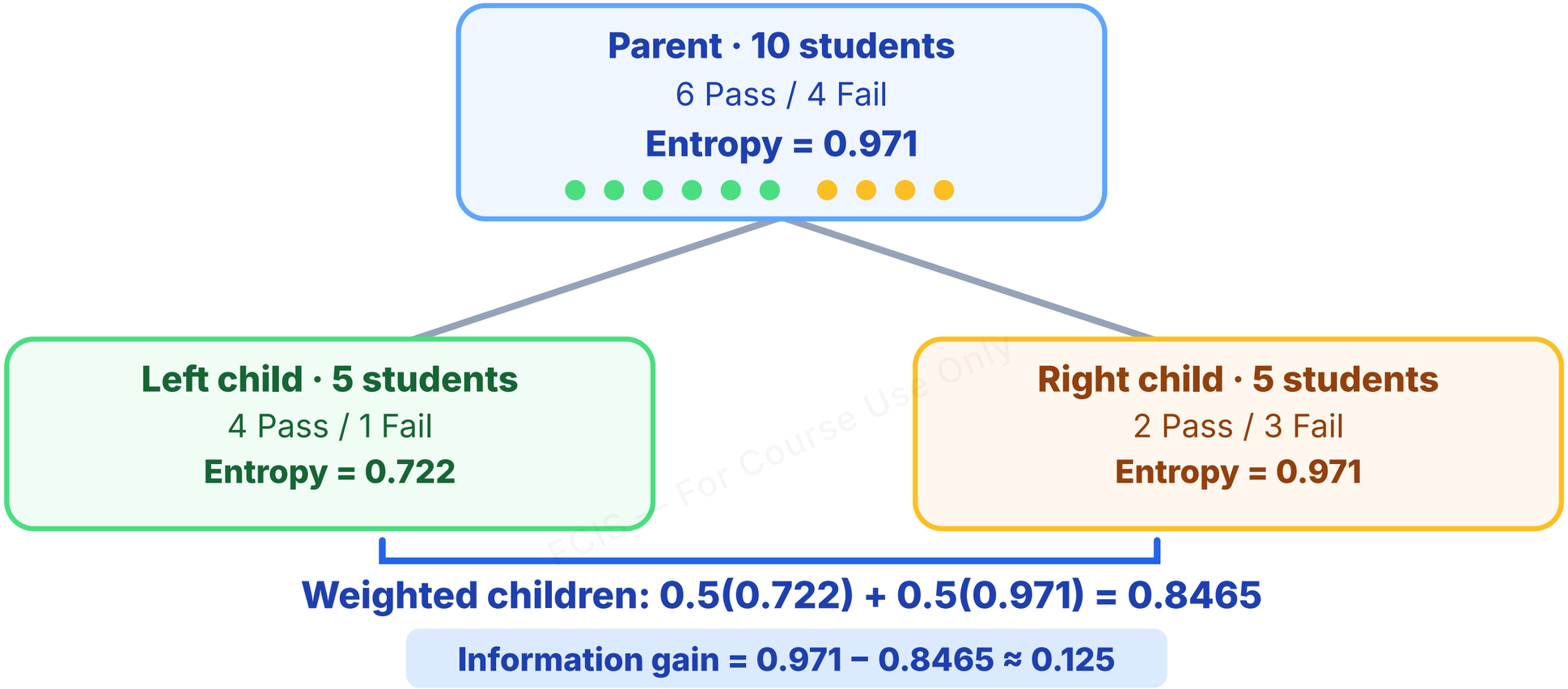
FCIS - For Course Use Only

Compare the parent to the weighted impurities of its children



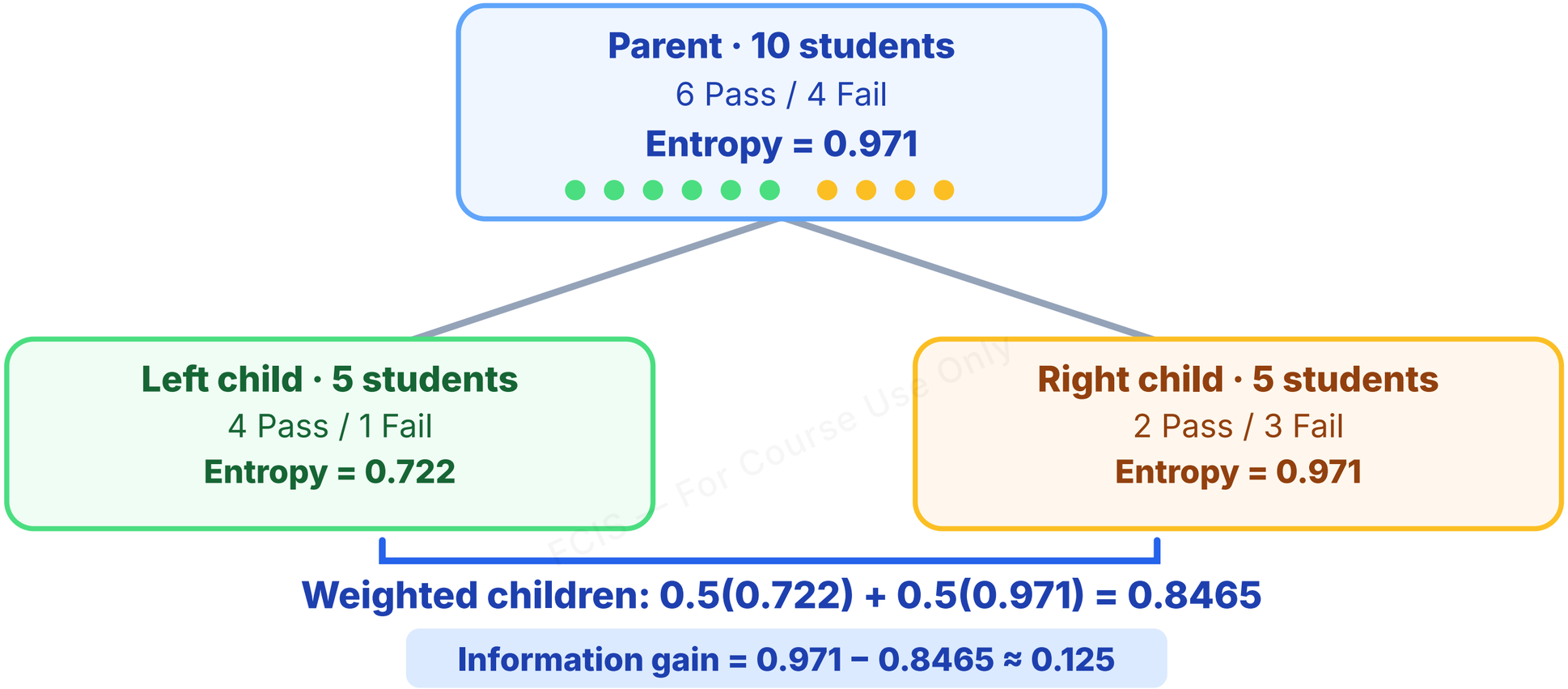
ECIS2 For Course Use Only

Compare the parent to the weighted impurities of its children



For Course Use Only

Compare the parent to the weighted impurities of its children



ECIS2 For Course Use Only

💡 **Takeaway.** A split is useful when the **weighted impurity of the children** is **lower** than the impurity of the **parent**. The larger the reduction, the larger the **information gain**.

Information Gain Calculation

FCIS — Private Use Only

Continue the example

Weighted child entropy:

$$\frac{5}{10}(0.722) + \frac{5}{10}(0.971) = 0.8465$$

Information gain:

$$IG = 0.971 - 0.8465 = 0.1245$$

Interpretation

The split reduces **entropy**, but not dramatically.

> Another candidate split with larger **information gain** would be preferred.

IUG — For Course Use Only

Gini Gain

FCIS — Private Use Only

Same idea with Gini impurity

Using **Gini**, the impurity reduction is:

$$Gain_{\text{Gini}} = Gini(S) - \sum_{j} \frac{|S_j|}{|S|} Gini(S_j)$$

A tree can select the split with the largest Gini reduction.

Practical note

Many implementations use Gini by default because it is computationally simple.

> The algorithmic idea remains the same: choose the split that **improves purity the most**.

FCIS — For Course Use Only

Numerical Feature Splits

FCIS — For Classroom Use Only

Thresholds

For numerical features, trees often use threshold **splits**.

Example:

$$\text{attendance} \leq 75?$$

This creates two children:

- examples with attendance ≤ 75
- examples with attendance > 75

FCIS — For Course Use Only

Candidate thresholds

Possible thresholds are often tested between sorted observed values.

> If attendance values are $60, 70, 80, 90$, candidate thresholds could include $65, 75, 85$.

Categorical Feature Splits

FCIS — For Course Use Only

Branches by category

Decision trees can split observations according to the **categories of a feature**.

Example: Learning Mode

Possible splitting conditions:

- **Single category:** Is Learning Mode = **Group**?
- **Group of categories:** Is Learning Mode in **{Group, Online}**?

Important

The splitting strategy depends on the algorithm:

- **Multiway split:** one branch per category.
- **Binary split:** categories divided into two groups.
- **One-hot encoding:** binary features can be used for splitting.

IS — For Course Use Only

Takeaway. Features with many categories offer more possible splits and may increase the risk of **overfitting**.

Part V

FCIS — For Private Use Only

Manual Tree Construction

To understand **decision trees**, students should be able to construct a small tree manually.

FCIS — For Course Use Only

Training Data

FCIS — Private Use Only

Small student-success dataset

Student	Attendance	Labs Completed	Final Result
S1	High	Yes	Pass
S2	High	Yes	Pass
S3	High	No	Pass
S4	Low	Yes	Fail
S5	Low	No	Fail
S6	Low	No	Fail
S7	High	No	Fail
S8	Low	Yes	Fail

Class distribution

Total: 8 students

- **Pass:** 3 students
- **Fail:** 5 students

Candidate features

- **Attendance**
- **Labs Completed**

Task

Which feature should define the **first split** of the decision tree?

Choose using **information gain**.

FCIS — For Course Use Only

Parent Entropy

FCIS — Academic Use Only

Before any split

The parent node has:

$$p_{\text{Pass}} = \frac{3}{8} \qquad p_{\text{Fail}} = \frac{5}{8}$$

Entropy:

$$H(S) = -\left[\frac{3}{8}\log_2\frac{3}{8} + \frac{5}{8}\log_2\frac{5}{8}\right]$$

$$H(S) \approx 0.954$$

FCIS — For Course Use Only

💡 This is the uncertainty before asking any feature question.

Candidate Split: Attendance

FCIS — Private Use Only

Compute child entropies

Split by `Attendance`:

Attendance	Pass	Fail	Total
High	3	1	4
Low	0	4	4

Child entropies

$$H(\text{High}) = -\left[\frac{3}{4}\log_2\frac{3}{4} + \frac{1}{4}\log_2\frac{1}{4}\right] \approx 0.811$$

$$H(\text{Low}) = 0 \qquad (\text{pure node})$$

FCIS — For Course Use Only

Weighted entropy and information gain

$$H_{\text{children}} = \frac{4}{8}(0.811) + \frac{4}{8}(0) = 0.4055$$

$$IG(S, \text{Attendance}) = 0.954 - 0.4055 = 0.5485$$

Candidate Split: Labs Completed

FCIS — For Course Use Only

Compute child entropies

Split by `Labs Completed`:

Labs Completed	Pass	Fail	Total
Yes	2	2	4
No	1	3	4

Child entropies

$$H(\text{Yes}) = 1 \qquad (\text{equal class proportions})$$

$$H(\text{No}) = -\left[\frac{1}{4}\log_2\frac{1}{4} + \frac{3}{4}\log_2\frac{3}{4}\right] \approx 0.811$$

FCIS Prof. Course Use Only

Weighted entropy and information gain

$$H_{\text{children}} = \frac{4}{8}(1) + \frac{4}{8}(0.811) = 0.9055$$

$$IG(S, \text{Labs}) \approx 0.954 - 0.9055 = 0.0485$$

Choose the Root Split

FCIS — University Use Only

Which feature wins?

Information gains:

Candidate split	Information gain
Attendance	0.5485
Labs Completed	0.0485

The root split is: `Attendance`

Why?

It reduces uncertainty much more than the `Labs Completed` split.

> The `Low` attendance branch is already pure: all examples are Fail.

For Course Use Only

Continue the Tree

FCIS — Lecture Use Only

Split only where needed

After splitting by `Attendance`:

Low → Fail (pure node) **High → Mixed** (3 Pass, 1 Fail)

The `High` node contains:

Student	Labs Completed	Result
S1	Yes	Pass
S2	Yes	Pass
S3	No	Pass
S7	No	Fail

FCIS — For Course Use Only

Now split the `High` node by `Labs Completed`:

Yes → Pass (pure node) **No → Mixed** (S3: Pass, S7: Fail)

> 💡 **Takeaway.** Split only where needed. **S3 and S7** have identical recorded features (**High**, **No**) but opposite outcomes, so these features cannot separate them into pure leaves.

Stopping Conditions

FCIS — Course Use Only

When should a tree stop growing?

A tree may stop when:

- a node is pure;
- no useful split remains (or no eligible feature test is available);
- maximum depth is reached;
- minimum samples per split is not satisfied;
- minimum samples per leaf is not satisfied;
- impurity decrease is too small;

For Course Use Only

Hyperparameters controlling these stopping rules are then **selected using validation or cross-validation**; validation examples are not used as ordinary training examples at each node.

Prediction at non-pure leaf

If a leaf still contains mixed labels, predict the **majority class** or class probabilities.

> Stopping early can reduce **overfitting** but may increase bias.

Part VI

FCIS — Lecture Use Only

Tree Complexity and Generalization

A **decision tree** can fit complex patterns, but deep trees can memorize the training data.

FCIS — For Course Use Only

Depth and Overfitting

FCIS — Private Course Use Only

Flexible trees can be too flexible

A **deep decision tree** can create many small leaves, sometimes containing only a few training examples.

Such trees may memorize **noise, outliers, rare coincidences, or data-collection artifacts** rather than learn general patterns.

Effect of tree depth

Tree complexity	Training error	Validation error
Too shallow	High	High
Suitable depth	Low / moderate	Low
Too deep	Very low	May increase

FCIS — For Course Use Only

How can we control overfitting?

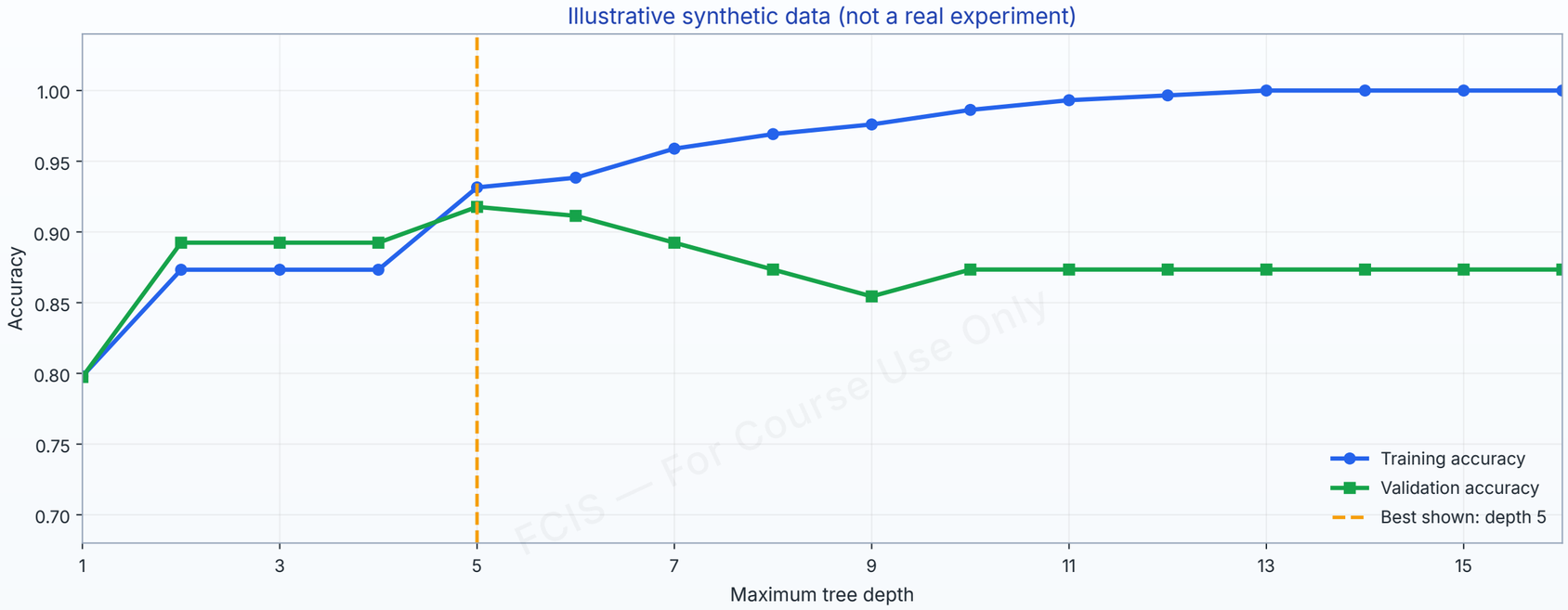
Limit **maximum depth**, require a **minimum number of samples per leaf**, or **prune** unnecessary branches.

> **Key idea.** A tree that perfectly classifies its training examples may still **overfit**. Choose tree complexity using **validation performance**, not training accuracy alone.

Plot: Tree Depth and Generalization

FCIS — For Course Use Only

A deeper tree can fit training noise without improving validation accuracy



Fixed synthetic dataset with a held-out validation split. Training accuracy increases with tree depth, while validation accuracy does not consistently improve.

Model selection. Prefer cross-validation when selecting tree depth, and reserve the **test set** for final evaluation.

Pre-Pruning

FCIS — For Course Use Only

Restrict growth while building the tree

Pre-pruning stops the tree before it becomes too complex.

Common controls:

Hyperparameter	Effect
`max_depth`	limits path length
`min_samples_split`	requires enough examples to split
`min_samples_leaf`	prevents tiny leaves
`max_leaf_nodes`	limits number of leaves
minimum impurity decrease	split only if improvement is large enough

FCIS — For Course Use Only

> These hyperparameters should be selected using validation or cross-validation.

Post-Pruning

FCIS — For Course Use Only

Grow first, then simplify

Post-pruning first grows a large tree and then removes branches that do not improve validation or penalized performance.

Basic idea

If replacing a subtree with a leaf does not harm validation performance, prune it.

Why useful?

- large trees may capture training-specific details;
- pruning can improve generalization;
- smaller trees are easier to interpret.

FCIS — For Course Use Only

> 💡 **Pruning** balances model complexity and predictive performance.

Tree Interpretability

FCIS — Course Use Only

Useful but not automatic

Decision trees can be interpretable, but interpretability depends on their complexity.

Easier to interpret when

- the tree is **small**;
- features are meaningful;
- splits are simple;
- paths are not too deep;
- leaves contain enough examples.

Harder to interpret when

- the tree has **hundreds of nodes**;
- features are encoded or transformed heavily;
- splits are unstable;
- small data changes produce different trees.

FCIS — For Course Use Only

A **simple tree** can explain a prediction through a short sequence of decision rules.

Feature Importance

FCIS — For Reference Use Only

Use with caution

Tree-based feature importance often measures **how much a feature reduces impurity** across splits.

Useful

- identifies features used strongly by the tree;
- helps summarize model behavior.

Caution

- can favor features with many possible splits;
- can be unstable;
- does not prove causality;
- depends on the training data and model settings.

FCIS — For Course Use Only

> Feature importance answers **“What did this model use?”** not “What causes the target?”

Part VII

FCIS — Students Use Only

K-Nearest Neighbors

KNN predicts a **new example** by looking at the **labels of nearby training examples**.

FCIS — For Course Use Only

KNN Intuition

FCIS — For Course Use Only

Similar examples should have similar labels

K-Nearest Neighbors (KNN) is an instance-based learning method.

Training: Store the labeled training examples.

Prediction:

1. Compute distances from the new example to the training examples.
2. Select the k **nearest neighbors**.
3. For **classification**, predict by majority vote.
4. For **regression**, predict by averaging the neighbors' targets.

FCIS — For Course Use Only

> **Takeaway.** KNN does not fit a compact global prediction formula. It relies on **stored training examples** when making predictions.

Distance Measure

FCIS — Course Private Use Only

Euclidean distance

For two points:

$$x = (x_1, \ldots, x_p)$$

and:

$$z = (z_1, \ldots, z_p)$$

Euclidean distance is:

$$d(x, z) = \sqrt{\sum_{j=1}^{p} (x_j - z_j)^2}$$

For Course Use Only

Interpretation

Smaller **distance** means more similar according to the feature representation.

> The meaning of “near” depends on preprocessing and feature choice.

KNN Classification Rule

FCIS — Private Use Only

Majority vote

Let $N_k(x)$ denote the indices of the k **nearest neighbors** of a new point x.

KNN classification predicts the class receiving the most votes:

$$\hat{y} = \arg\max_c \sum_{i \in N_k(x)} \mathbb{1}(y_i = c)$$

Here, $\mathbb{1}(y_i = c)$ equals **1** if neighbor i belongs to class c, and **0** otherwise.

Example: $k = 5$

Neighbor labels: Pass · Pass · Fail · Pass · Fail

Vote counts: - **Pass:** 3 votes - **Fail:** 2 votes

Predicted class: $\hat{y} = \text{Pass}$ (majority of votes).

For Course Use Only

KNN Regression Rule

FCIS Course Use Only

Average of neighbor targets

For regression, KNN predicts the mean target among the **nearest neighbors**:

$$\hat{y} = \frac{1}{k} \sum_{i \in N_k(x)} y_i$$

Example

If the nearest rent values are:

$$1800,\ 2000,\ 2100$$

FCIS — For Course Use Only

then:

$$\hat{y} = \frac{1800 + 2000 + 2100}{3} = 1966.67$$

> **KNN** can be used for both classification and regression.

Worked KNN Example

FCIS — Personal Use Only

Distances in two dimensions

A new student has **Hours = 5** and **Attendance = 75**:

$$x = (5, 75)$$

Using **Euclidean distance**, calculate the distance to each training example:

Point	Hours	Attendance	Class	Distance to x
A	2	60	Fail	$\sqrt{(5-2)^2 + (75-60)^2} = \sqrt{234} \approx 15.30$
B	4	70	Fail	$\sqrt{(5-4)^2 + (75-70)^2} = \sqrt{26} \approx 5.10$
C	7	80	Pass	$\sqrt{(5-7)^2 + (75-80)^2} = \sqrt{29} \approx 5.39$
D	8	85	Pass	$\sqrt{(5-8)^2 + (75-85)^2} = \sqrt{109} \approx 10.44$

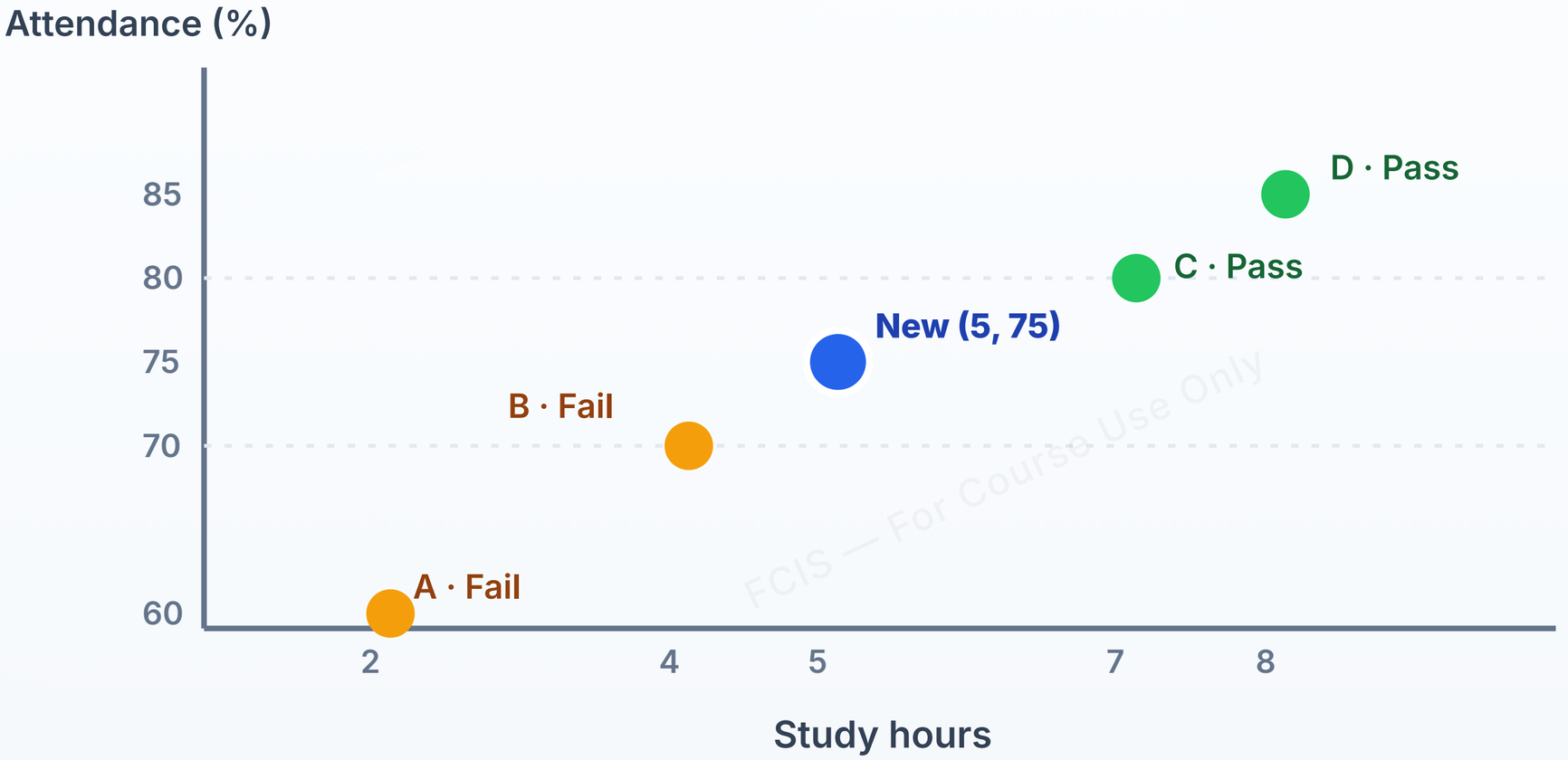
Nearest neighbors (in order): B → C → D → A.

FCIS — For Course Use Only

Visual: KNN Finds Its Neighbors

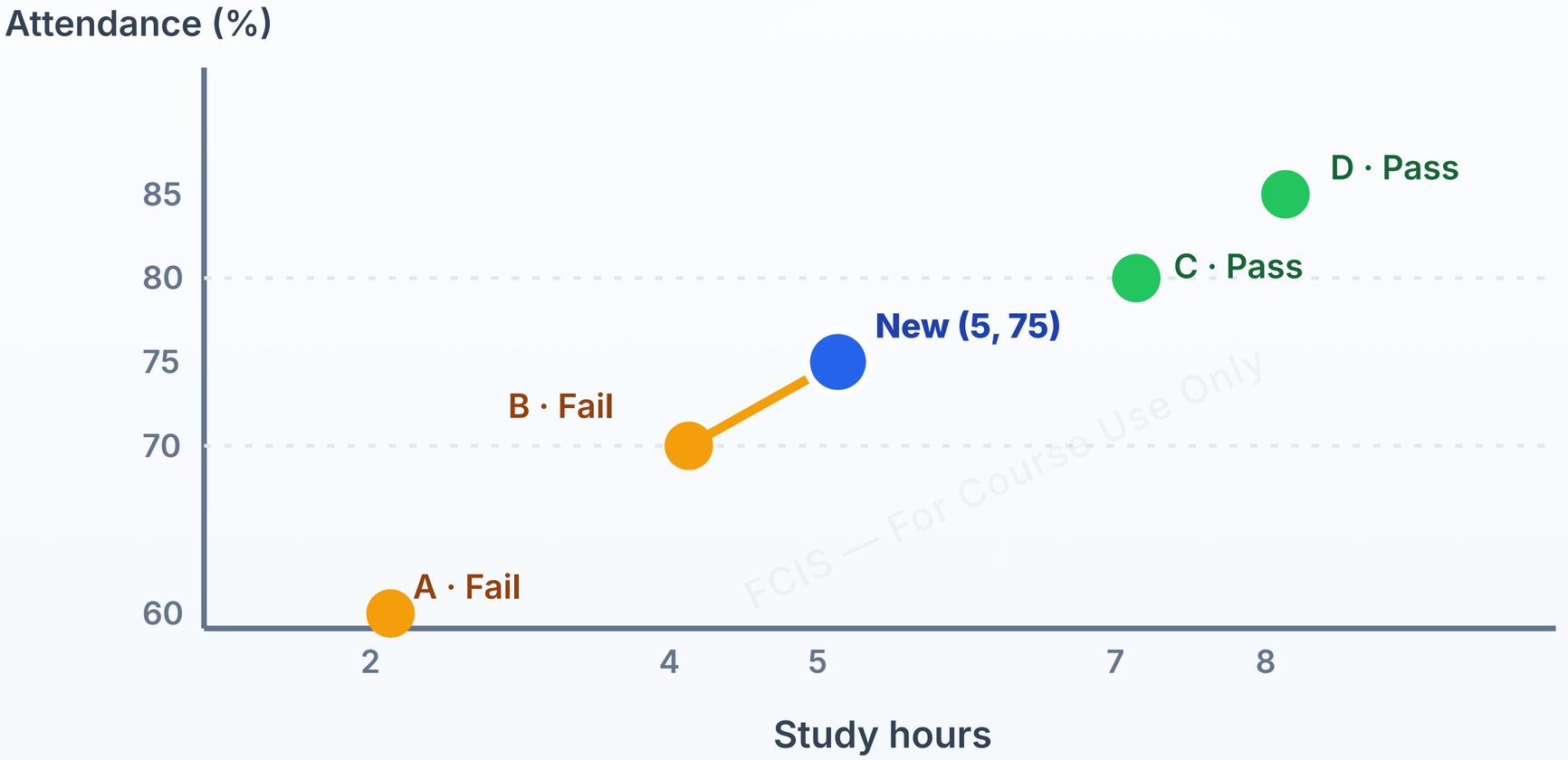
FCIS — Private Use Only

Four known students and one new student



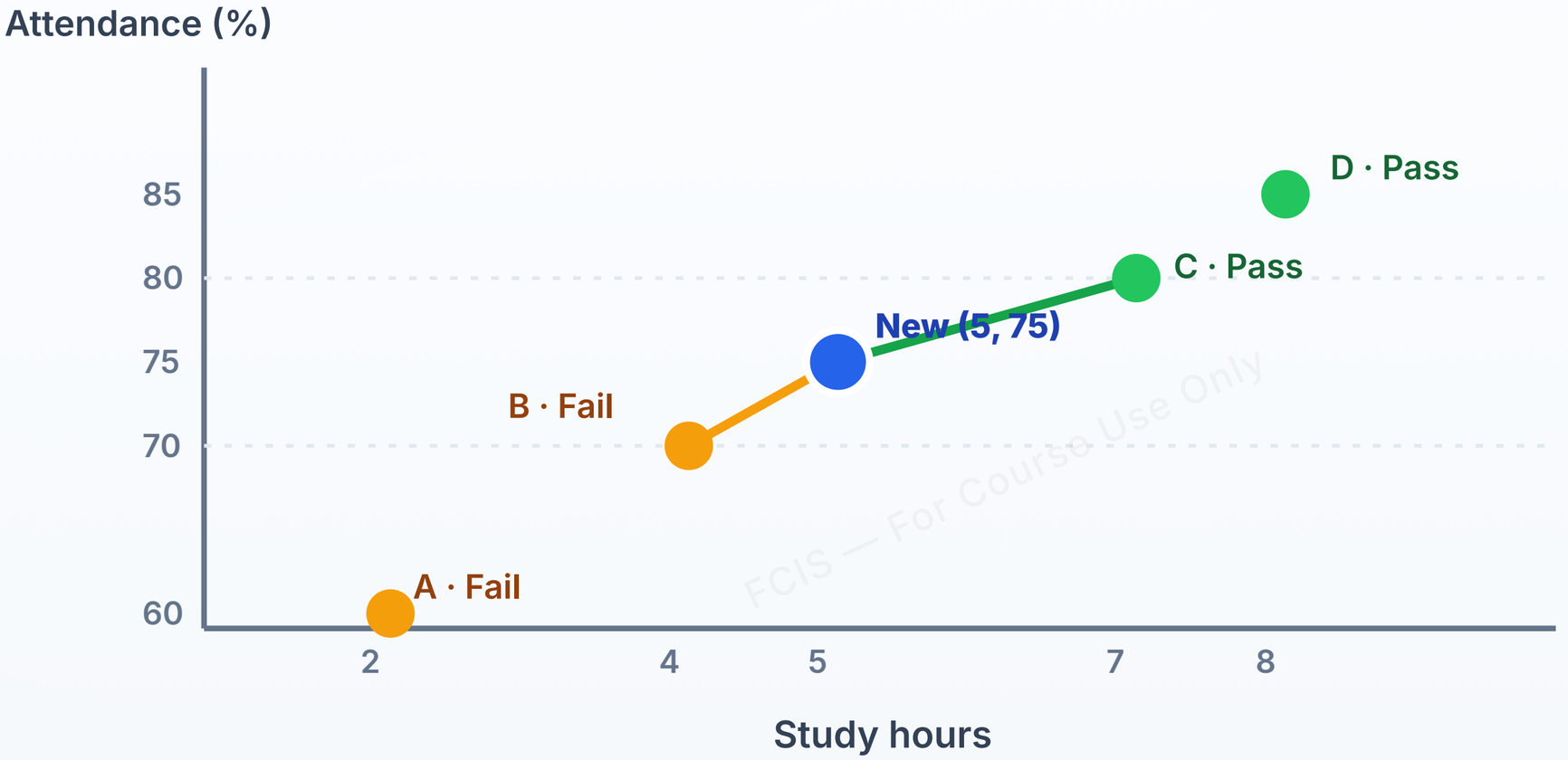
FCIS — For Course Use Only

Four known students and one new student



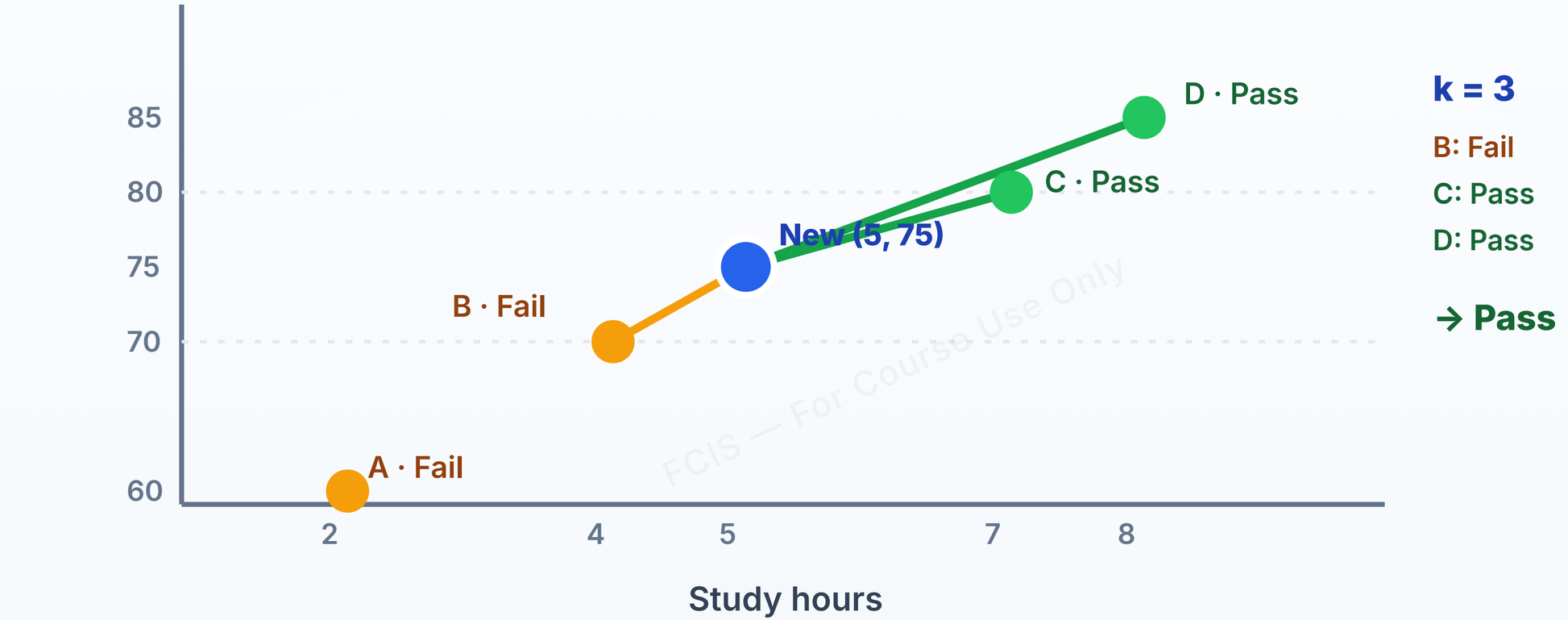
FCIS — For Course Use Only

Four known students and one new student



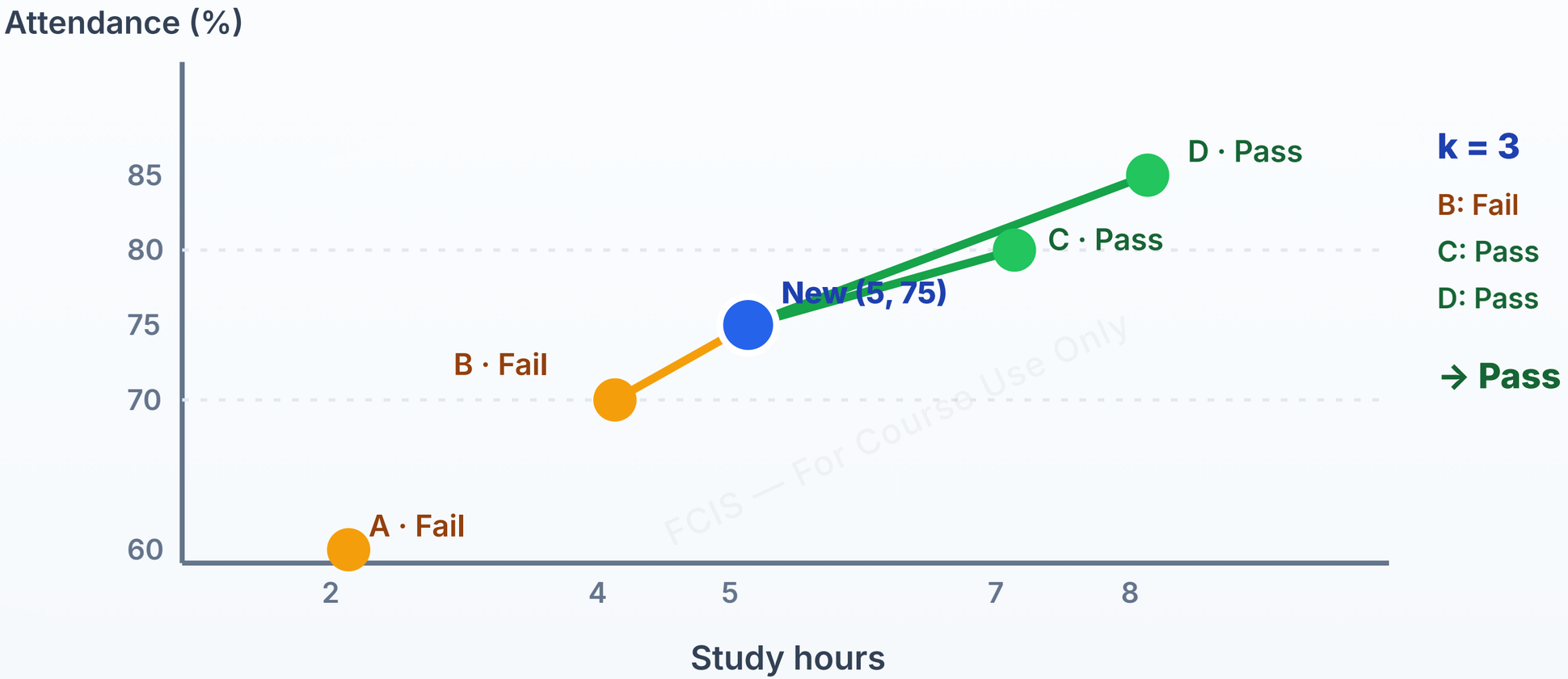
FCIS — For Course Use Only

Four known students and one new student



FCIS — For Course Use Only

Four known students and one new student



FCIS — For Course Use Only

💡 **Takeaway.** For $k = 3$, the nearest neighbors are **B (Fail)**, **C (Pass)**, and **D (Pass)**. The majority vote predicts **Pass**. Distances are calculated using unscaled feature values; the drawing uses different axis scales, so visual line lengths can be misleading.

Worked KNN Example

FCIS — Internal Use Only

Prediction with $k = 3$

Nearest neighbors:

Rank	Point	Distance	Class
1	B	5.10	Fail
2	C	5.39	Pass
3	D	10.44	Pass

For: $k = 3$,

the majority class is: `Pass`.

FCIS — For Course Use Only

> 💡 The **KNN** prediction depends directly on the selected value of k and the feature scales.

Choosing k

FCIS — Personal Use Only

Local sensitivity vs. smoothness

The number of neighbors k controls how local or smooth KNN predictions are.

Choice	Behavior	Main risk
Small k	More flexible; sensitive to nearby examples	**Overfitting** to noise
Large k	Smoother; uses a broader neighborhood	**Underfitting** local patterns

Extreme case: $k = 1$

The model predicts the **label of the single nearest training example**.

This can fit training data very closely but may **generalize poorly** to unseen observations.

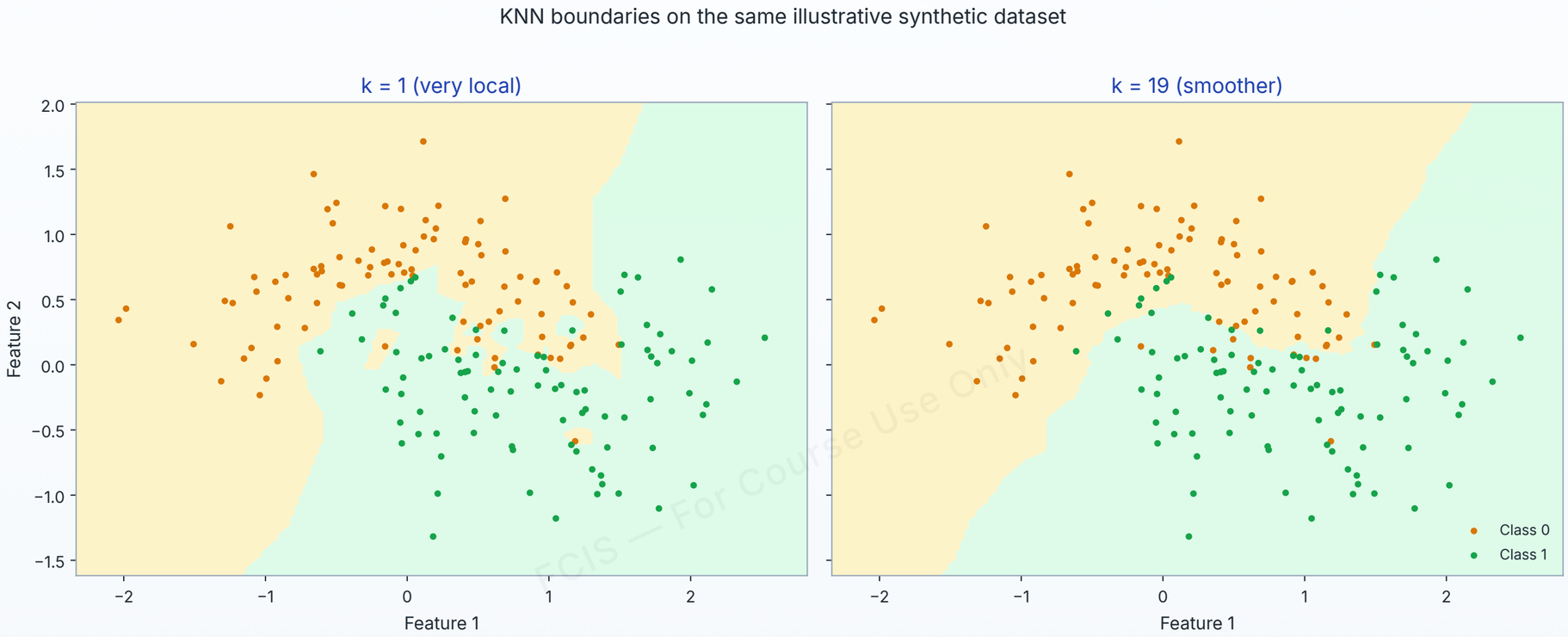
> 💡 **Key idea.** Choosing k balances **local flexibility and smoothness**. Use **validation or cross-validation** to find a suitable value.

FCIS — Course Use Only

Plot: Changing k Changes the Decision Boundary

Property Of For Course Use Only

Local details versus a smoother neighborhood rule



Key caution. Changing k can substantially alter the **decision boundaries**. Choose k using **validation or cross-validation**, not the final test set.

Tie Handling

FCIS — For Course Use Only

What if neighbor votes are equal?

KNN classification can produce ties when multiple classes receive the same number of votes.

Example: With $k = 4$, the neighbors vote: `Pass`, `Pass`, `Fail`, `Fail` → **2–2 tie**

Common tie-breaking strategies

- **Odd k:** Avoids equal votes in unweighted binary classification.
- **Nearest neighbors:** Favor the tied class with closer neighbors.
- **Distance-weighted voting:** Give closer neighbors more influence.
- **Class priors:** Use prior class frequencies to resolve ties.
- **Deterministic rule:** Apply a consistent fallback decision.

FCIS — For Course Use Only

Distance-weighted voting

One possible weight for neighbor i is:

$$\text{weight}_i = \frac{1}{d_i + \epsilon}$$

where d_i is its distance and $\epsilon > 0$ prevents division by zero.

💡 **Key idea.** Majority voting treats all neighbors equally; **distance-weighted voting** gives more influence to nearby observations.

Why Scaling Matters for KNN

FCIS - For Course Use Only

Distance can be dominated by large-scale features

Suppose features are:

Feature	Typical range
Income	20,000 to 300,000
Age	18 to 80
Number of visits	0 to 20

In Euclidean distance:

$$d(x, z) = \sqrt{\sum_j (x_j - z_j)^2}$$

FCIS — For Course Use Only

income differences may dominate simply because income has larger numerical units.

Solution

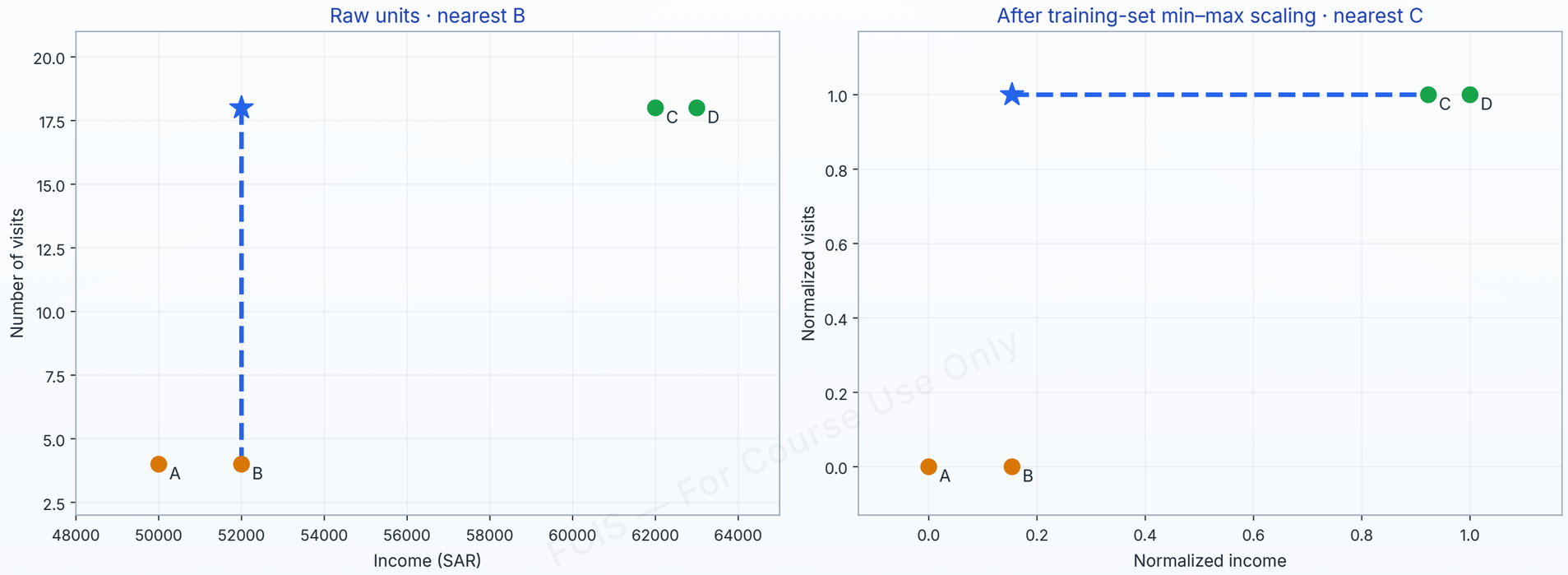
Standardize or normalize features before using KNN. **Fit the scaler on the training portion only** and apply it unchanged to validation/test data (or fit it separately inside each cross-validation training fold).

> For **KNN**, preprocessing is part of the model's definition of similarity.

Plot: Scaling Can Change the Nearest Neighbor

For Course Use Only

A distance-based prediction depends on units



For — For Course Use Only

💡 **Takeaway.** With $k = 1$, raw units select **B (Fail)**, while min–max scaling selects **C (Pass)**. Because KNN depends on distances, **feature scaling can change predictions**. Fit the scaler on training data only.

KNN and Dimensionality

FCIS — Private Use Only

The curse of dimensionality

As the number of features increases, **distance-based similarity can become less reliable**, especially when features are noisy or irrelevant.

Challenges

- **Sparse data:** Points spread out in high-dimensional space.
- **Less informative distances:** Nearest and farthest points may have similar distances.
- **Misleading neighbors:** Irrelevant features can obscure useful similarities.
- **More data needed:** Local patterns become harder to estimate reliably.

Practical Responses

- **Scale** numerical features.
- **Select** informative features.
- **Reduce dimensionality** when appropriate.
- **Use domain knowledge** to engineer useful representations.
- **Validate** k and preprocessing choices.

FCIS — For Course Use Only

Key caution. KNN relies on meaningful distances. It can work well with **informative feature representations**, but may struggle with high-dimensional, noisy data.

Part VIII

FCIS — Lecture Use Only

Trees vs. KNN

Decision trees learn **feature-based decision rules**, while **KNN** makes predictions from **nearby training examples**.

Both solve supervised-learning problems, but they rely on **different assumptions about the data**.

FCIS — For Course Use

Comparison Table

FCIS — Course Use Only

How Do They Differ?

Aspect	Decision Tree	KNN
Learning approach	Learns decision rules	Stores training examples
Training effort	Builds a tree	Minimal (lazy learning)
Prediction effort	Usually fast	Can be costly for large datasets
Feature scaling	Usually not needed	Important for distances
Interpretability	High for small trees	Based on nearby examples
Decision boundaries	Nonlinear, axis-aligned splits	Nonlinear, distance-based
Key hyperparameters	Depth, leaf size, split criterion	k, distance metric
Overfitting risk	Very deep trees	Very small k
Best suited when	Feature-based rules are meaningful	Local similarity is meaningful

FCIS - For Course Use Only

Model Selection

FCIS — For Educational Use Only

Which should we choose?

Choosing an algorithm depends on both **predictive performance** and **application requirements**.

Predictive suitability

- validation performance;
- data size;
- feature types;
- noise level.

Practical requirements

- interpretability needs;
- scaling requirements;
- prediction-time constraints;
- application costs.

Example: Decision tree vs. KNN

- **Decision tree:** A small tree may be preferable when **clear decision rules** and interpretability are important.
- **KNN:** May work well when **similarity between examples** is meaningful and features are appropriately scaled.

For Course Use

💡 **Takeaway.** There is **no universally best algorithm**. Compare models using validation performance while considering the requirements and costs of the application.

Practice & Consolidation

FCIS — For Course Use Only

Reasoning About Trees and Neighbors

Practice **impurity**, **information gain**, nearest neighbors, and model evaluation.

FCIS — For Course Use Only

Activity: Compute Impurity

FCIS — Personal Use Only

Compare node purity

A decision-tree node contains **8 positive** and **2 negative** examples.

Tasks

1. Calculate the **entropy** of the node.
2. Calculate its **Gini impurity**.
3. Is this node more or less pure than one containing **5 positive** and **5 negative** examples?

FCIS — For Course Use Only

Compare node purity

A decision-tree node contains **8 positive** and **2 negative** examples.

Tasks

1. Calculate the **entropy** of the node.
2. Calculate its **Gini impurity**.
3. Is this node more or less pure than one containing **5 positive** and **5 negative** examples?

Solution. The class proportions are $p_+ = 0.8$ and $p_- = 0.2$.

- **Entropy:** $H = -0.8 \log_2(0.8) - 0.2 \log_2(0.2) \approx 0.722$.
- **Gini:** $G = 1 - 0.8^2 - 0.2^2 = 0.32$.
- **Comparison:** This node is **more pure** than a 5/5 node, whose entropy is 1 and Gini is 0.5.

FCIS Course Use Only

Activity: Choose a Split

FCIS — Classroom Use Only

Compare information gain

The parent contains **5 positive** and **5 negative** examples, so its entropy is $H(S) = 1$.

Two candidate splits produce:

Candidate	Child	Positive	Negative
A	Left	4	1
	Right	1	4
B	Left	3	2
	Right	2	3

FCIS — For Course Use Only

Task

Which split should the tree **prefer** based on information gain? Explain why.

Compare information gain

The parent contains **5 positive** and **5 negative** examples, so its entropy is $H(S) = 1$.

Two candidate splits produce:

Candidate	Child	Positive	Negative
A	Left	4	1
	Right	1	4
B	Left	3	2
	Right	2	3

ECIS — For Course Use Only

Task

Which split should the tree **prefer** based on information gain? Explain why.

Solution. Both candidates create equally sized children.

$$IG_A = 1 - [0.5(0.722) + 0.5(0.722)] \approx 0.278$$
$$IG_B = 1 - [0.5(0.971) + 0.5(0.971)] \approx 0.029$$

Choose A: It produces **purer children** and has greater information gain.

Activity: KNN Scaling

FCIS — For Class Use Only

Predict the problem

A KNN model uses:

- income in SAR;
- age in years;
- number of visits.

No scaling is applied.

Questions

1. Which feature may dominate **Euclidean distance**?
2. Why is this not necessarily meaningful?
3. What preprocessing step should be applied?
4. Would a **decision tree** be equally sensitive to this scaling issue?

FCIS — For Course Use Only

Predict the problem

A KNN model uses:

- income in SAR;
- age in years;
- number of visits.

No scaling is applied.

Questions

1. Which feature may dominate **Euclidean distance**?
2. Why is this not necessarily meaningful?
3. What preprocessing step should be applied?
4. Would a **decision tree** be equally sensitive to this scaling issue?

FCIS — For Course Use Only

> Income may dominate due to its numeric scale. Standardization or normalization should be applied. A tree is usually much less sensitive to monotonic scaling.

Activity: Tune Hyperparameters

FCIS — For Reference Use Only

Validation results

Model	Hyperparameter	Training Accuracy	Validation Accuracy
Tree	depth = 2	0.78	0.76
Tree	depth = 8	0.99	0.71
KNN	$k = 1$	1.00	0.73
KNN	$k = 7$	0.86	0.82

Questions

1. Which models likely overfit?
2. Which model should be selected by validation accuracy?
3. Why should the test set not be used here?

ECIS — For Course Use Only

Validation results

Model	Hyperparameter	Training Accuracy	Validation Accuracy
Tree	depth = 2	0.78	0.76
Tree	depth = 8	0.99	0.71
KNN	$k = 1$	1.00	0.73
KNN	$k = 7$	0.86	0.82

Questions

1. Which models likely overfit?
2. Which model should be selected by validation accuracy?
3. Why should the test set not be used here?

Tree depth 8 and KNN $k = 1$ likely overfit. KNN $k = 7$ has best validation accuracy. Test data must remain for final evaluation.

ECIS — For Course Use Only

Common Mistakes

FCIS — Personal Use Only

Trees and KNN

Mistake	Why it matters
Growing trees until every leaf is pure	Can lead to **overfitting**
Reporting accuracy without tree complexity	Hides depth and **overfitting risk**
Treating feature importance as causation	Importance does **not prove causality**
Using KNN without feature scaling	Large-scale features may **dominate distances**
Selecting k using test performance	Causes **test-set leakage**
Including irrelevant features in KNN	Makes distances less informative
Assuming an interpretable tree is correct	**Interpretability ≠ validity**
Ignoring class imbalance	Accuracy may hide **minority-class errors**

FCIS — For Course Use Only

Mini-Quiz

FCIS — For Course Use Only

Check your understanding

1. What does a decision tree leaf represent?
2. What does node impurity measure?
3. Why does **information gain** prefer some splits over others?
4. Why can deep trees overfit?
5. Why is feature scaling less critical for trees than for KNN?
6. What does k control in KNN?
7. Why can $k = 1$ overfit?
8. Why can KNN struggle in high-dimensional data?

For Course Use Only

Check your understanding

1. What does a decision tree leaf represent?
2. What does node impurity measure?
3. Why does **information gain** prefer some splits over others?
4. Why can deep trees overfit?
5. Why is feature scaling less critical for trees than for KNN?
6. What does k control in KNN?
7. Why can $k = 1$ overfit?
8. Why can KNN struggle in high-dimensional data?

For Course Use Only

Suggested answers: a final prediction or class distribution; label mixture; reduction in uncertainty; deep trees can fit noise; trees split by thresholds while KNN uses distances; k controls neighborhood size; one neighbor is noise-sensitive; distances become less informative in high dimensions.

Lecture 6 Summary

FCIS — For Course Use Only

Key ideas to retain

Decision Trees

- **Trees** predict by following feature-based decisions from the root to a leaf.
- Classification leaves typically predict the **majority class**.
- **Entropy and Gini** measure node impurity on different scales.
- **Information gain** measures the reduction in weighted impurity after a split.
- Trees naturally capture **nonlinear relationships and feature interactions**.
- **Deep trees** can overfit; control complexity using depth limits, minimum leaf sizes, or pruning.

K-Nearest Neighbors (KNN)

- **KNN** predicts using nearby training examples.
- A **distance measure** determines which observations are neighbors.
- **Small k:** More sensitive to noise; **large k:** Smoother predictions.
- **Feature scaling** matters because distance depends on numerical units.
- KNN supports both **classification** (voting) and **regression** (averaging).
- Use **validation** to select k and other hyperparameters, just as for tree complexity.

FCIS — For Course Use Only

End-of-Lecture Check

FCIS — Private Use Only

Five questions to test your understanding

1. How does a **decision tree** choose a split using entropy, Gini, or information gain?
2. Why can a deep tree achieve **perfect training accuracy** but perform poorly on unseen data?
3. How can decision trees capture **feature interactions** without explicitly creating interaction features?
4. How does **KNN** classify a new observation, and what happens when k changes?
5. Why is **feature scaling** important for KNN, and how can we avoid preprocessing leakage?

For Course Use Only

💡 **Preparation for Lecture 7.** Review **weighted sums, logistic regression, activation functions, gradient descent, and the chain rule**. Next, we combine these ideas to understand how **neural networks** compute predictions and learn from errors.

Suggested Reading

FCIS — Private Use Only

Primary and supporting references

Primary reference

G. James, D. Witten, T. Hastie, and R. Tibshirani, *An Introduction to Statistical Learning with Applications in Python*, Springer, 2023.

- Review sections on tree-based methods, classification, model flexibility, and validation.
- Review KNN as a simple non-parametric supervised learning method.

Supporting references

A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, O'Reilly, 2022.

- Review decision trees, model tuning, and practical classification workflows.

K. P. Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022.

- Use selected sections for deeper views of classifiers, loss, uncertainty, and model complexity.

FCIS For Course Use Only

💡 When reading, focus on how the model makes decisions internally and why validation is needed to control complexity.